GOVERNMENT OF INDIA — COPYRIGHT OFFICE

ANNEXURE TO FORM XIV (RULE 70) — STATEMENT OF TECHNICAL PARTICULARS

LEXIS

Technical Supporting Documentation & System Specification

Accompanying Source Code Deposit for Computer Software Work

STATEMENT OF PARTICULARS — COMPUTER SOFTWARE WORK

1. Title of Work:	LEXIS (Autonomous Multi-Agent Legal Aid System for Indian Citizens)
2. Class / Category:	Computer Software [Literary Work under Section 2(o) of Copyright Act, 1957]
3. Language of Work:	Python, JavaScript (React JSX), CSS, HTML, Shell (Human Language: English, Hindi)
4. Country of Origin:	India
5. Applicant / Intended Owner:	Tejas Vikas Chaudhari
6. Authors & Contributors (5 Total):	1. Tejas Vikas Chaudhari 2. Pratik Pandurang Devadhe 3. Manjit Ajit Bajaj 4. Vedant Jitendra Gaikwad 5. Dr. Anil Kumar Brahamane (Project Guide & Mentor)
7. Repository Location:	https://github.com/TejasC005/LEXIS
8. Core Frameworks:	Backend: FastAPI 0.115 & LangGraph 1.2 | Frontend: React 18.3 (Vite 5.4)
9. Primary Data Stores:	MongoDB (Async Motor), Redis (Session Cache), Qdrant (Vector Engine)

STATUTORY OWNERSHIP & AUTHORSHIP WARNING:

Ownership and authorship details must be verified against the actual agreement/assignment/NOC before submission. Under Section 17 of the Indian Copyright Act, 1957, absent an express assignment deed or NOC, all 5 contributing authors remain joint first owners. If filed in sole name of applicant, a formal No Objection Certificate (NOC) signed by the 4 co-authors is mandatory.

PURPOSE & SCOPE OF THIS TECHNICAL SUPPORTING DOCUMENT

This technical document provides the necessary architectural, structural, and descriptive details of the LEXIS software to assist the Copyright Office in examining the original computer software work. It establishes the nature of the software as an original literary work comprising asynchronous agent state machines, algorithmic coordination, database ingestion pipelines, cryptographic vault management, and responsive user interfaces. IMPORTANT NOTICE: Under the rules and guidelines of the Indian Copyright Office, the official software deposit consists of the source code document 'LEXIS_Copyright_Source_Code.pdf' (containing the First 10 and Last 10 pages of the original source code). This supporting document serves strictly as an explanatory technical annexure and must NOT be presented as a replacement for the mandatory source code deposit.

TABLE OF CONTENTS OF SUPPORTING ANNEXURE

Section 1: Executive Summary & Project Purpose — Page 2
Section 2: Multi-Agent Architecture & LangGraph State Machine — Page 3
Section 3: The Six Specialist Agents & Bounded Reflection Layer — Page 4
Section 4: Technical Stack, Storage & Retrieval Pipelines — Page 5
Section 5: Security Architecture, Cryptography & Access Control — Page 6
Section 6: Source Code Repository Inventory & Exclusion Register — Page 7
Section 7: Statutory Originality & Authorship Declaration — Page 8

1. EXECUTIVE SUMMARY & SYSTEM PURPOSE

1.1 The Citizen Legal Aid Problem in India

Every year, millions of Indian citizens face consumer disputes, landlord-tenant conflicts, unfair contract terms, digital scams, and statutory filing dilemmas without accessible, timely, or affordable legal assistance. While the Indian legal framework grants robust rights under acts such as the Right to Information Act (2005), Consumer Protection Act (2019), and state tenancy enactments, citizens face substantial barriers: procedural opacity, complex legal terminology, linguistic barriers, and severe delays. Furthermore, existing generic generative AI tools often produce 'hallucinated' legal citations, invent sections of non-existent acts, and fail to provide actionable, grounded guidance.

1.2 The LEXIS Solution: Autonomous Multi-Agent Systems

LEXIS is an autonomous multi-agent legal aid software platform designed and developed specifically for Indian citizens. Rather than relying on a monolithic prompt-and-response model, LEXIS implements a distributed cognitive architecture orchestrated by LangGraph. In LEXIS, multiple specialized software agents collaborate to solve complex legal problems: one agent plans and decomposes intents, specialized agents execute domain tasks concurrently, a dedicated confirmation agent verifies outputs against statutory checklists, and a background scheduling service proactively protects citizen deadlines.

1.3 Core Functional Pillars of LEXIS

- **Pillar 1 — Rights Explanation:** Grounds legal queries in statutory data using dense vector embeddings (BGE-M3) and Qdrant retrieval, requiring exact citation verification before output.
- **Pillar 2 — Document Drafting:** Guides citizens through dynamic, step-by-step questionnaires to produce court-ready and authority-ready legal drafts (RTI applications, legal notices, consumer complaints) with dual PDF and DOCX export.
- **Pillar 3 — Contract & Clause Analysis:** Ingests user agreements (PDF, PNG, JPEG), executes OCR via PyMuPDF and Tesseract, and classifies e clause into Green/Yellow/Red risk tiers with bilingual Hindi/English explanations.
- **Pillar 4 — Scam Detection & Pattern Grounding:** Cross-references fraudulent messages, phishing notices, and extortive schemes against a 17-pattern vector database, eliminating speculative generation.
- **Pillar 5 — Regulatory Filing Authority Guidance:** Identifies the exact jurisdictional forum, online portal, or nodal officer for citizen complaint automatically ingesting drafts generated by the Drafting Agent without re-entry.
- **Pillar 6 — Proactive Encrypted Vault & Deadline Management:** Stores sensitive citizen records using authenticated AES-256-GCM encrypt asynchronous scheduler that triggers proactive notifications and notice drafting at T-30, T-15, and T-3 days.
- **Pillar 7 — Bounded Self-Correction & Contradiction Detection:** A dedicated Confirmation Agent inspects all generated outputs, detects cr (Rule C1), and enforces a 3-retry cap before delivering an honest fallback.

2. CORE ARCHITECTURE: LANGGRAPH MULTI-AGENT STATE MACHINE

2.1 The Computational Model: Graph State Machine vs. Sequential if/else

Traditional web backends handle multi-step tasks sequentially using procedural if/else chains. If a user asks a composite question—such as 'Explain my tenant rights under the Model Tenancy Act and draft an eviction reply notice'—a procedural script would run the rights analyzer, wait for completion, and then execute the drafter. This results in linear latency (T_total = T_rights + T_drafter) and brittle state coupling. LEXIS solves this through a directed cyclic graph state machine implemented with LangGraph (v1.2).

2.2 The Shared State: OrchestratorState

The backbone of the LEXIS execution lifecycle is OrchestratorState (defined in backend/agents/orchestrator/state.py). It is a TypedDict containing the citizen query, historical conversation turns, detected intent flags, validation records, and a shared scratchpad. A critical innovation is the scratchpad reducer: Annotated[List[AgentOutput], operator.add]. This reducer guarantees that when multiple agents run in parallel and return output simultaneously, LangGraph combines their results via list addition rather than letting the last finishing agent overwrite the state.

2.3 Concurrent Fan-Out Routing via the Send API

When an intent classification node (classify_intent_node) identifies multiple legal intents, the fan_out_router dynamically produces a list of Send objects: [Send('specialist_node', {'intent': i, ...}) for i in intents]. LangGraph schedules all matched specialist agents in the same execution superstep. Because the agents execute in parallel, total processing latency is bounded by max(T_1, T_2, ... T_n) rather than their cumulative sum, delivering an immediate 2x to 3x responsiveness boost.

2.4 Partial Failure Isolation

In distributed multi-agent systems, external API or service failures are inevitable. Under LangGraph's atomic execution rules, an unhandled exception in any parallel node crashes the entire superstep. LEXIS implements strict error isolation wrappers: if a specialist agent encounters a timeout or dependency fault, it catches the exception internally and returns an AgentOutput marked with status='error' and an honest fallback. The compile_response_node merges successful agent results while transparently notifying the citizen of partial degradations.

2.5 Architectural Workflow Diagram

The complete execution topology follows a five-stage progression: (1) Ingestion & Session Memory Hydration (Redis) -> (2) Dynamic Intent Decomposition -> (3) Concurrent Fan-Out Superstep (Specialists) -> (4) Confirmation & Cross-Agent Contradiction Checking (Module 11) -> (5) Unified Response Compilation & Session Store Persistence.

3. SPECIALIST AGENTS & THE CONFIRMATION LAYER

- **Agent 1 — Rights Explainer (Module 5):** Acts as a legal research consultant for citizens. It combines BGE-M3 dense semantic embeddings w Qdrant vector retrieval across an ingested corpus of Indian statutes and consumer frameworks. Employs a deterministic citation check: if a generated answer references an Act or Section not grounded in the retrieved context, the output is rejected.
- **Agent 2 — Document Drafter (Module 6):** Transforms legal knowledge into concrete formal instruments. Employs modular templates for RT Applications, Section 138 Negotiable Instruments Notices, Consumer Protection complaints, and Tenancy disputes. Manages multi-turn questionnaire state and renders court-ready PDF (via ReportLab) and DOCX (via python-docx) files.
- **Agent 3 — Document Analyzer (Module 7):** Assists citizens in reviewing agreements, employment offers, and loan documents before signin secure multi-part file ingestion with MIME allowlisting and magic-byte inspection. Combines PyMuPDF for digital text extraction and Tesseract for scanned images, feeding extracted clauses into a hybrid rule-based and LLM-assisted risk classifier.
- **Agent 4 — Scam Detection Engine (Module 8):** Protects citizens against cyber fraud, electricity bill scams, courier delivery extortion, and fa summons. Grounded in a 17-pattern vector database stored in Qdrant and MongoDB. Utilizes regex keyword fast-paths and cosine similarity to deliver factual, warning-level alerts.
- **Agent 5 — Complaint Filing Guide (Module 9):** Solves the 'Where do I file?' dilemma. Houses a structured catalog of Indian authorities (e-Da Consumer Forums, RTI Online, RBI Ombudsman). Integrates directly with the Document Drafter: citizens can transition directly from draft creation to filing instructions without re-entering details.
- **Agent 6 — Vault & Deadline Agent (Module 10):** Provides AES-256-GCM encrypted document storage on the host filesystem. Features proa detection: an APScheduler daily job queries upcoming deadlines and automatically invokes the Document Drafter at T-30, T-15, and T-3 days to prepare proactive renewal letters or extension requests.
- **Confirmation Agent & Bounded Reflection (Module 11):** Acts as an automated QA legal editor. Reviews outputs against concrete checklist statutory citations, and cross-agent contradictions (Rule C1: Legal Act Mismatch). If validation fails, it triggers a bounded retry loop (capped at 3 iterations) before falling back.

4. TECHNICAL STACK & STORAGE ARCHITECTURE

4.1 Comprehensive Technology Inventory

LAYER / ROLE	TECHNOLOGY / STACK	VERSION	EXACT FUNCTIONALITY IN REPOSITORY
Component	Technology / Library	Version	Exact Purpose in LEXIS
Backend Framework	FastAPI / Uvicorn	0.115.0 / 0.32.0	Asynchronous REST API, lifespan events, middleware
Multi-Agent Engine	LangGraph / LangChain	1.2.11 / 1.6.2	State machine, intent routing, concurrent fan-out
Data Modeling	Pydantic / Settings	2.12.5 / 2.5.2	Type enforcement, request validation, config parsing
Primary Database	MongoDB / Motor	4.9.2 / 3.6.0	Async persistence of users, drafts, logs, seed catalogs
Session Memory	Redis Async Client	5.2.0	Fast session state, rate limit counters, scratchpad cache
Vector Search	Qdrant Client / FastEmbed	1.19.0 / 0.8.0	Vector embeddings (BGE-M3) for RAG & scam similarity
Document Parsing	PyMuPDF / Tesseract	1.28.2 / 0.3.13	PDF digital parsing & scanned image optical character recognition
Document Export	ReportLab / python-docx	5.0.1 / 1.2.0	Court-ready PDF generation and editable DOCX compilation
Job Scheduler	APScheduler	3.10.4	Async daily cron trigger for proactive deadline reminders
Frontend Client	React 18 / Vite	18.3.1 / 5.4.8	Component-based SPA, rapid bundling, responsive design
Client Routing	React Router DOM	6.27.0	Protected route guards, nested layouts, client navigation
HTTP Client	Axios	1.7.7	REST requests, automatic JWT injection & interceptors
Cryptography	cryptography (Python)	43.0.1	AES-256-GCM symmetric encryption for Vault files

4.2 Storage Tiering & Document Flow Architecture

LEXIS implements strict storage tiering: (1) Transient Session Memory: Ephemeral conversation turns and intent states reside in Redis under keys formatted as lexis:session:{session_id}, expiring automatically to maintain privacy; (2) Persistent Metadata & Relational State: User profiles, document records, and validation audit logs reside in MongoDB; (3) Dense Semantic Knowledge: Statutory rights passages and scam patterns are indexed in Qdrant collections with 1024-dimensional BGE-M3 vectors; (4) Encrypted Document Vault: Physical files uploaded by citizens are stored at VAULT_STORAGE_PATH/{user_id}/{file_uuid} encrypted using AES-256-GCM.

5. SECURITY ARCHITECTURE, CRYPTOGRAPHY & ACCESS CONTROL

5.1 Authenticated Encryption at Rest (AES-256-GCM)

Citizen legal documents (contracts, tenancy agreements, notices) contain highly sensitive personal data. In backend/security/vault_crypto.py, LEXIS implements authenticated AES-256-GCM (Galois/Counter Mode) encryption using the Python cryptography library. Every file is encrypted with a unique 96-bit (12-byte) initialization vector (IV / nonce). The ciphertext and authentication tag are packed into the binary file header. Tampering with any byte of ciphertext or metadata results in immediate cryptographic decryption failure, preventing both data leakage and unauthorized alteration.

5.2 Authentication & Role-Based Access Control (RBAC)

Authentication is handled via a custom OTP provider supporting phone and secondary email delivery (integrated with Brevo REST API). Upon successful verification, the system issues an ephemeral JWT access token (15-minute validity) and a refresh token (7-day validity). User roles are strictly segregated via FastAPI dependency guards (require_role):

- **Citizen Role ('user'):** Full access to conversational agents, document drafting, risk analysis, and personal encrypted vault.
- **Administrator Role ('admin'):** Access to system analytics, reflection logs, and user status controls, isolated behind require_role(['admin']).
- **Lawyer Role ('lawyer'):** Scaffolded verification role (Module 13) isolated behind require_role(['lawyer']) with strict mutual 403 authorization barriers.
- **Anonymous Mode:** Allows public access to preliminary rights inquiries while strictly blocking persistent vault operations.

5.3 Prompt Injection Defense & AI Safety

AI agents interacting with citizen inputs face adversarial prompt injection risks. LEXIS implements a multi-layer defense (backend/security/prompt_injection_defense.py and backend/api/middleware/prompt_injection_guard.py): (1) Heuristic Pattern Matching: Detects jailbreak signatures ('ignore previous instructions', 'system override', 'DAN mode'); (2) Input Sanitization & Token Delimitation: Encapsulates user inputs within strict boundaries in specialist prompts; (3) Confirmation Agent Review: Validates outputs before delivery, discarding responses that break out of the legal domain.

5.4 File Upload Hardening

Upload endpoints enforce three strict security checks: (1) Extension allowlisting (.pdf, .jpg, .jpeg, .png); (2) Magic-byte signature inspection (ensuring file contents match declared MIME types, blocking polyglot executables); and (3) Strict file size limits (10 MB cap).

6. REPOSITORY INVENTORY & STATUTORY EXCLUSIONS REGISTER

6.1 Source Code Inventory by Programming Language

LANGUAGE / EXTENSION	FILE COUNT	TOTAL LINES	PRIMARY FUNCTION IN LEXIS
Language / Extension	File Count	Total Lines of Code	Primary Role in Codebase
Python (.py)	122 files	18,572 lines	FastAPI routes, LangGraph agents, security, services, models
JavaScript / React (.jsx, .js)	31 files	5,009 lines	React 18 components, pages, custom hooks, API service layer
CSS Stylesheets (.css)	15 files	4,909 lines	Responsive UI styling for chat, analyzer, vault, dashboards
JSON Seed Data (.json)	7 files	4,620 lines	Statutory rights seed, scam pattern vectors, filing authorities
HTML Template (.html)	1 file	37 lines	Single Page Application (SPA) DOM mounting root
Shell Script (.sh)	1 file	9 lines	Developer environment bootstrap utility (scripts/setup.sh)
Total Original Work	177 source files	33,156 lines	Complete custom application codebase written by authors

6.2 Statutory Exclusions Register (Security & Non-Original Code)

Pursuant to standard copyright filing practice and confidentiality obligations, the following files and directories have been strictly excluded from the deposited materials:

- **Environment Secrets (backend/.env):** Contains active production secret keys, JWT salts, and private API credentials.
- **Service Account Keys (backend/firebase/service_account.json):** Contains private RSA signing keys for Google/Firebase authentication.
- **Third-Party Node Modules (frontend/node_modules/):** External npm packages (React, Vite, Axios, etc.) written by third-party authors.
- **Virtual Environment (backend/venv/):** Standard Python interpreter binary and compiled third-party dependencies.
- **Python Bytecode & Cache (__pycache__/, .pytest_cache/):** Machine-compiled binary cache files generated during test execution.
- **Compiled Build Assets (frontend/dist/):** Minified and bundled JavaScript/CSS build artifacts generated by Vite.
- **Dependency Lockfiles (frontend/package-lock.json):** Machine-generated dependency tree manifests containing no original logic.
- **Encrypted User Storage (vault_storage/, backend/vault_storage/):** Encrypted test user files and binary uploads containing dummy citizen data.

7. STATUTORY AUTHORSHIP, OWNERSHIP & VERIFICATION DECLARATION

7.1 Complete Authorship & Contribution Matrix (5 Authors)

The computer software work entitled 'LEXIS' was authored and developed through collaborative engineering by the following five contributors:

- **1. Tejas Vikas Chaudhari (Applicant & Contributing Co-Author):** Multi-agent architecture, LangGraph state machine, backend services, API rout
- **2. Pratik Pandurang Devadhe (Contributing Co-Author):** Frontend architecture, React component development, UI/UX, email OTP integration.
- **3. Manjit Ajit Bajaj (Contributing Co-Author):** Document analysis pipeline, OCR integration, clause risk classification heuristics.
- **4. Vedant Jitendra Gaikwad (Contributing Co-Author):** Specialist agent workflows, Qdrant vector ingestion, scam pattern seed cataloging.
- **5. Dr. Anil Kumar Brahamane (Contributing Co-Author & Project Guide):** Project supervision, architectural guidance, legal compliance verificat

7.2 Statutory Ownership Notice & NOC Requirement (Section 17, Copyright Act, 1957)

Under Section 17 of the Indian Copyright Act, 1957, in the absence of an employer-employee agreement or written assignment deed, all five contributing authors remain joint first owners of copyright. Where Tejas Vikas Chaudhari applies as the sole applicant/owner on Form XIV, a formal No Objection Certificate (NOC) cum Assignment Deed signed by the remaining four co-authors (Pratik Pandurang Devadhe, Manjit Ajit Bajaj, Vedant Jitendra Gaikwad, and Dr. Anil Kumar Brahamane) is required to be submitted to the Copyright Office.

7.3 Pre-Submission Verification Checklist for Form XIV Filing

- **[ ] Source Code Deposit Document::** Ensure 'LEXIS_Copyright_Source_Code.pdf' (21 pages) is uploaded under the Software Deposit section.
- **[ ] Supporting Technical Annexure::** Submit 'LEXIS_Copyright_Supporting_Document.pdf' as an explanatory technical annexure to Form XIV.
- **[ ] Verification of Contributor Names::** Verify that all 5 author names match exactly with official government identity documents (Aadhaar / PAN).
- **[ ] Execution of 4-Person NOC::** If sole ownership is claimed by applicant, ensure the 4-Person NOC is signed, notarized, and attached.
- **[ ] Zero Secrets Confirmation::** Verify that no API keys, private passwords, or active .env variables are present in any deposited material.

DECLARATION BY APPLICANT:

I hereby declare that the particulars given above are true and that the software work deposited is an original creation.

Signature of Applicant: _____ **Date:** _____

(Tejas Vikas Chaudhari) Place: Kopargaon, Maharashtra, India